

TP — Stateful Web Tracking

Imane Fouad | UM6P

Mission: Follow Alice

Alice visits a website without creating an account and without logging in. The website still wants to recognize her when she returns and to build a small profile of the pages she visits. In this practical session, your job is to build a prototype that tracks Alice's activity and to test different stateful tracking mechanisms. At every step, you will investigate what is stored in the browser, what is transmitted in HTTP, and what the server can learn.

How can a stateless HTTP server recognize the same browser across multiple requests?

You will work in two roles:

- **Web developer:** create the website and its tracking mechanisms.
- **Privacy investigator:** inspect what the browser and server exchange.

Note

For each challenge, follow the same investigation cycle:

Predict → Test → Inspect → Explain

Collect evidence from both the **browser Developer Tools** and the **Flask server terminal**.

1 Setting Up the Environment

1.1 Create the Working Folder

Create a folder named `statefulTracking`, then open PowerShell in that folder.

This folder will contain the server code, web pages and resources used during the practical session.

1.2 Create a Python Virtual Environment

A virtual environment, or `venv`, is an isolated Python environment. It lets us install the packages required for this TP without changing the computer's main Python installation.

Create and activate it:

```
python -m venv venv
venv\Scripts\activate
```

After activation, `(venv)` should appear before the command prompt.

If PowerShell blocks the activation script, run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
venv\Scripts\activate
```

This permission change applies only to the current PowerShell session.

1.3 Install the Web Server

Flask is a lightweight Python framework for creating web servers. It will let us define pages and directly manipulate HTTP requests, responses and cookies.

Install Flask inside the activated virtual environment:

```
pip install flask
flask --version
```

Note

If you are already familiar with another server technology, you may use it instead. Your implementation must reproduce the same routes, headers and cookie behaviour required in the challenges.

2 Running Your First Website

2.1 Create the Project Structure

Inside `statefulTracking`, create an `app.py` file and a `templates` folder containing `index.html`:

```
statefulTracking/
|-- venv/
|-- app.py
|-- templates/
    |-- index.html
```

2.2 Create the Webpage

Add the following code to `templates/index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Alice's Web Journey</title>
</head>
<body>
  <h1>Welcome to Alice's Web Journey</h1>

</body>
</html>
```

The HTML file defines what the browser displays. For now, this page is static: it does not create or read cookies.

2.3 Create the Flask Server

Add the following code to `app.py`:

```
from flask import Flask, render_template
```

```
app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)
```

This program:

- creates a Flask web application;
- associates the path / with the function `home()`;
- loads `index.html` from the `templates` folder;
- starts a local HTTP server on port 8000;
- automatically reloads the server when the code changes.

2.4 Start the Server

From the `statefulTracking` folder, run:

```
python app.py
```

The terminal should display a message similar to:

```
Running on http://127.0.0.1:8000
```

Keep this terminal open. It is your **server-side observation point**.

2.5 Open the Website in your browser

In your browser, open:

```
http://127.0.0.1:8000
```

Here, `127.0.0.1` is the local computer and `8000` identifies the service running on it. The browser is the client and Flask is the server.

Domain names are easier to recognize than IP addresses. We will associate the name `lab.test` with `127.0.0.1` using the Windows hosts file.

Open PowerShell **as Administrator** and run:

```
notepad C:\Windows\System32\drivers\etc\hosts
```

Add this line at the end of the file, then save it:

```
127.0.0.1 lab.test
```

Now open the website using:

```
http://lab.test:8000
```

Note

If you cannot edit the hosts file, you may continue with `http://127.0.0.1:8000`, but use that address consistently.

3 Warming Up — Inspecting HTTP Headers

The browser sends **request headers** to describe its request. The server returns **response headers** that describe the result.

Update `app.py` with the following code:

```
from flask import Flask, render_template, request, make_response

app = Flask(__name__)

@app.route("/")
def home():
    response = make_response(render_template("index.html"))

    # Add a custom header to the HTTP response
    response.headers["X-Lab-Message"] = "Hello from the Flask server"

    # Display the request and response headers in the server terminal
    print("\n--- HTTP REQUEST HEADERS ---")
    print(request.headers)

    print("--- HTTP RESPONSE HEADERS ---")
    print(response.headers)

    return response

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)
```

Your Task

Open the browser Developer Tools and select the **Network** panel, then examine:

- the **request headers** sent by the browser;
- the **response headers** returned by the server;
- the headers displayed in the Flask terminal.

4 Warming up — Manipulating Cookies

HTTP is stateless: each request is processed independently, and the server does not automatically remember previous requests.

To recognize Alice when she returns, the website needs to store a cookie containing a unique identifier in her browser.

Update `app.py` by adding the following imports:

```
from flask import Flask, render_template, request, make_response
```

```
import secrets

app = Flask(__name__)

@app.route("/")
def home():
    # Check whether the browser already has an identifier
    aid = request.cookies.get("aid")
    is_new = aid is None

    # Generate an identifier only for a new browser
    if is_new:
        aid = secrets.token_hex(8)

    # Create the normal HTTP response containing index.html
    response = make_response(render_template("index.html"))

    # Ask the browser to store the identifier
    if is_new:
        response.set_cookie(
            key="aid",
            value=aid
        )

    return response

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)
```

Open the browser Developer Tools and inspect the stored cookie.

Close and restart the browser, then return to the website.

Question: Is the cookie still present? Does it have the same identifier?

Your Task

Update `app.py` to test the different cookie attributes introduced during the course. After each modification:

1. delete the cookie created during the previous test;
2. reload the website to create a new cookie;
3. inspect the cookie in Developer Tools;
4. observe how the attribute changes its storage or behaviour;
5. close and restart the browser when testing persistence;
6. record and explain your observations.

5 Challenge 1 — Third-Party Tracking

5.1 Introducing Third-Party Cookies

A cookie is considered **third-party** when it is created or sent in a context where the cookie's domain is different from the website displayed in the browser's address bar.

In this challenge, you will create:

- two independent publisher websites;
- one third-party tracker;
- an iframe that loads the tracker inside both publishers.

Add these entries to the Windows hosts file:

Create the following project structure:

```
statefulTracking/  
|-- publisher-one/  
|   |-- app.py  
|   |-- templates/  
|       |-- index.html  
|  
|-- publisher-two/  
|   |-- app.py  
|   |-- templates/  
|       |-- index.html  
|  
|-- tracker-one/  
|   |-- app.py  
|   |-- templates/  
|       |-- tracker.html
```

Each publisher must display different content. For example:

- Publisher 1: technology articles;
- Publisher 2: travel articles.

Create a tracker page and include it in both publishers using an iframe. The query parameter should allow the tracker to determine which publisher or page caused the request.

Your Task

Update the three applications so that:

1. both publisher websites run independently;
2. each publisher embeds content from `tracker 1`;
3. Tracker 1 assigns the browser a unique persistent identifier;
4. Tracker 1 stores that identifier in a cookie;
5. Tracker 1 records every received request;
6. the tracker log associates the identifier with the visited publisher and page.

From the server side, present the reconstructed profile in chronological order.

6 Challenge 2 — First-Party Analytics

Analytics services often provide JavaScript that publishers include in their pages exp:

```
<script src="http://analytics.test:9100/static/analytics.js"></script>
```

Although the file comes from `analytics.test`, the script executes in the context of the publisher page. It can therefore create and read cookies belonging to the publisher, unless they are marked `HttpOnly`.

Use different hosts for this experiment. In the analytics website, create an `analytics.js` file and include it in both publisher.

The script must:

1. check whether an analytics cookie exists;
2. generate a random identifier if it does not exist;
3. store the identifier using `document.cookie`;
4. keep the identifier for future page visits.

Visit each publisher and inspect the cookies stored in the browser. Record the value and domain of each cookie.

Questions:

- Are the two publishers assigned the same identifier or different identifiers?
- Why does this behaviour occur?

On the analytics server, inspect the requests received from each publisher.

Question: Does the analytics server automatically receive the first-party cookies created in the publishers' contexts? Explain your observation.

Your Task

Following the analytics approach introduced in the course, update `analytics.js` to collect the user's browsing activity and send it to the analytics server. Include the same analytics script in a second publisher website. Then:

1. inspect the cookies stored for each publisher;
2. inspect the analytics requests in Developer Tools;
3. inspect the requests received by the analytics server;
4. reconstruct the profile(s) collected by the analytics service.

Determine whether the analytics service creates one combined profile or multiple separate profiles, and explain the result.

7 Optional Challenge — Cookie Syncing

7.1 Scenario

Two independent advertising trackers observe Alice: `tracker-one.test` identifies her as A123; and `tracker-two.test` identifies her as B789.

Because cookies are isolated by domain, neither tracker can directly read the other tracker's cookie. However, they can cooperate to learn that A123 and B789 belong to the same browser. This process is called **cookie syncing**.

Build Two Trackers and implement cookie syncing to synchronize Their Identifiers